

Ecole Polytechnique de Sousse	
Révision Examen	
<i>Classe : 1 LGL</i> <i>Matière : ASD 2</i> <i>Enseignant : Sakka Rouis Taoufik</i> <i>Documents Autorisés : Non</i>	<i>Session principale</i> <i>A.U. : 2025_2026</i> <i>Durée : 2H 00</i> <i>Nombre Total de Pages : 1</i>



Exercice 1 :

Soit la déclaration de liste chaînée suivante :

Types

Cellule = Struct

 cle : entier

 suiv : ^Cellule

FinStruct

Liste = Struct

 premier: ^Cellule

 dernier : ^Cellule

FinStruct

Implémenter les sous-programmes suivants :

1- Procédure creerListe (var L1 : Liste)

Cette procédure permet de créer une liste linéaire vide.

2- Fonction listeVide (L1 : Liste) : boolean

Cette fonction permet de vérifier si la liste L1 est vide ou non.

3- Procédure concatListe (L1 : Liste ; L2 : Liste ; Var L3 : Liste)

Cette procédure concatène les deux listes L1 et L2 dans une troisième liste L3.

4- Fonction adresseElement (L1 : Liste ; x : entier) : ^Cellule

Cette fonction rend l'adresse de la première cellule portant la valeur x.

5- Procédure SuppElement (var L1 : Liste ; x : entier)

Cette procédure supprime la première cellule contenant l'élément x dans la liste L1.

6- Procédure SuppTElement (var L1 : Liste ; x : entier)

Cette procédure supprime tous les cellules contenant l'élément x dans la liste L1.

7- Procédure DecoupeListe (L1 : Liste ; Var Lp : Liste ; Var Li : Liste)

Cette procédure coupe une liste d'entiers L1 en deux listes **Lp** contenant les entiers pairs, et **Li** contenant les entiers impairs.

8- Procédure inseret (var L1 : Liste ; x : entier)

Cette procédure insère (au bon endroit) l'élément x dans la liste L1 supposée déjà ordonnée par ordre croissant.

Ecole Polytechnique de Sousse	
Révision Examen	
<i>Classe : 1 LGL</i> <i>Matière : ASD 2</i> <i>Enseignant : Sakka Rouis Taoufik</i> <i>Documents Autorisés : Non</i>	<i>Session principale</i> <i>A.U. : 2025_2026</i> <i>Durée : 2H 00</i> <i>Nombre Total de Pages : 1</i>



Exercice 1 :

On veut écrire un algorithme qui gère l'ordre de décollage des avions d'un aéroport en fonction de l'ordre de leur horaire de décollage. On suppose que les avions qui souhaitent décoller sont mémorisés dans une liste chaînée simple de type **Lavion**.

Un **Avion** est représenté par son **numéro**, son **nom** de sa compagnie et l'**horaire** du vol. L'**horaire** du vol est représenté par l'**heure** et les **minutes** (par exemple : 9 h et 15 mn). Un Avion est inséré dans la liste au moment où il demande à pouvoir accéder à la piste. Il reste dans la liste tant qu'il y a des avions qui vont décoller avant lui. Enfin, l'avion est extrait de la liste lorsqu'il décolle.

Soit les enregistrements suivants :

Types

Horaire= Struct

Heure : entier

Minute : entier

FinStruct

Avion= Struct

Numero : entier

Nom : chaine

HoraireVol : Horaire

FinStruct

1. Définir les enregistrements « Cellule » et «Lavion» permettant la déclaration d'une liste chaînée d'avions.
2. Ecrire un sous algorithme SaisieAvion qui crée un nouvel avion A.
3. Ecrire un sous algorithme InsertAvion qui ajoute un Avion A à la liste des avions qui sont en attente de décoller. Noter bien que l'avion A doit insérer dans la bonne position pour que la liste L reste triée par ordre croissant de leur horaire.
4. Ecrire un sous algorithme Decoller qui supprime l'avion qui a décollé de la liste L.

5. Ecrire un sous algorithme AffichListe qui affiche les avions qui sont dans la liste L.